Meeting Assistant - System Spezifikation

🎯 Projektziel

Das Meeting Assistant System soll eine umfassende Lösung für die Live-Transkription und Verwaltung von Meetings bereitstellen. Das System besteht aus einer Chrome Extension für die Audio-Aufnahme und einem Backend-System für die Verarbeitung, Speicherung und Verwaltung der Transkripte.

Hauptziele:

- **Live-Transkription:** Echtzeit-Transkription von Meeting-Audio mit hoher Qualität
- **Meeting-Management:** Strukturierte Verwaltung von Meetings und deren Transkripten
- **Projekt-basierte Isolation:** Vollständige Tenant-Isolation pro Projekt
- **Suchfunktionalität:** Durchsuchbare Transkripte für einfache Nachbereitung
- **Skalierbarkeit:** System soll für verschiedene Meeting-Größen und -Typen geeignet sein
- **Benutzerfreundlichkeit:** Intuitive Bedienung über Chrome Extension

🏗️ Systemarchitektur

Komponenten:

1. **Screenity Integration** - Open-Source Screen Recorder für Video- und Audio-Aufnahme
2. **Chrome Extension** - Benutzer-Interface und Projekt-Management
3. **Backend-Server** - API-Server für Datenverwaltung
4. **Datenbank** - PostgreSQL für strukturierte Datenspeicherung
5. **Transkriptions-Service** - Integration mit Whisper API
6. **Audio-Verarbeitung** - Filterung und Optimierung der Audio-Daten

📋 Funktionale Anforderungen

1. Projekt-Integration

- **Externe Projekt-Verwaltung:** Projekte werden in externem System verwaltet
- **Projekt-Synchronisation:** Abruf von Projekt-Daten über externe API
- **Projekt-Validierung:** Verifizierung der Projekt-Berechtigungen
- **Projekt-Caching:** Lokales Caching von Projekt-Metadaten für Performance
- **Projekt-Export:** Export von Meeting-Daten für externe Systeme

2. Meeting-Management

Workflow für geplante Meetings:

1. Benutzer wählt Projekt aus externem System
2. System validiert Projekt-Berechtigung über externe API
3. Benutzer erstellt Meeting mit Titel und Beschreibung im Kontext des Projekts
4. Meeting wird mit `type: 'planned'` und `project_id` gespeichert
5. Benutzer wählt Meeting aus und startet Aufnahme
6. Alle Transkripte werden dem geplanten Meeting zugeordnet

Workflow für spontane Meetings:

1. Benutzer wählt Projekt aus externem System
2. System validiert Projekt-Berechtigung über externe API
3. Benutzer startet Screenity-Aufnahme ohne Meeting-Auswahl
4. System erstellt automatisch Meeting mit `type: 'spontaneous'` im ausgewählten Projekt
5. Auto-generierter Titel basiert auf Zeitstempel und Aufnahme-Kontext
6. Meeting kann später umbenannt und bearbeitet werden
7. System schlägt Zusammenführungen mit ähnlichen Meetings im gleichen Projekt vor

Workflow für Meeting-Zusammenführung:

1. System erkennt ähnliche Meetings innerhalb des gleichen Projekts (Zeit, Kontext, Teilnehmer)
2. Benutzer erhält Zusammenführungsvorschläge (nur innerhalb des Projekts)
3. Benutzer wählt Quell- und Ziel-Meeting aus
4. Alle Transkripte werden zum Ziel-Meeting migriert
5. Quell-Meetings werden als `status: 'merged'` markiert
6. Merge-Historie wird im Ziel-Meeting gespeichert

2. Audio-Aufnahme

- **Meeting erstellen:** Neue Meetings mit Titel und Beschreibung anlegen
- **Spontane Meetings:** Automatische Erstellung von Meetings beim Start der Aufnahme ohne Vorbereitung
- **Meeting auflisten:** Übersicht aller verfügbaren Meetings
- **Meeting abrufen:** Detaillierte Ansicht eines Meetings mit allen Transkripten
- **Meeting löschen:** Meetings und zugehörige Daten entfernen
- **Meeting bearbeiten:** Titel und Beschreibung aktualisieren

- **Meeting-Zusammenführung:** Mehrere spontane Meetings zu einem geplanten Meeting zusammenführen
- **Meeting-Identifikation:** Automatische Erkennung ähnlicher Meetings für Zusammenführungsvorschläge

3. Screenity-basierte Aufnahme

- **Vollständige Screen-Aufnahme:** Tab, Desktop, Anwendungen oder Kamera
- **Audio-Aufnahme:** Mikrofon und interne Audio-Aufnahme
- **Video-Export:** MP4, GIF, WebM Format-Unterstützung
- **Annotationen:** Zeichnungen, Text, Pfeile und Formen während der Aufnahme
- **Cursor-Highlighting:** Klick- und Cursor-Hervorhebung
- **Zoom-Funktionen:** Smooth Zoom für Fokus auf bestimmte Bereiche
- **Blur-Funktionen:** Verschleierung sensibler Inhalte
- **Automatische Stopps:** Timer und Alarm-Funktionen
- **Offline-Funktionalität:** Lokale Speicherung ohne Internetverbindung

4. Transkription

- **Live-Transkription:** Echtzeit-Verarbeitung der Audio-Daten
- **Mehrsprachigkeit:** Unterstützung verschiedener Sprachen
- **Qualitätsoptimierung:** Halluzinations-Erkennung und -Behandlung
- **Sprecher-Erkennung:** Automatische Identifikation und Zuordnung von Sprechern
- **Metadaten:** Speicherung von Zeitstempel, Dauer und Audio-Typ

5. Suchfunktionalität

- **Volltext-Suche:** Durchsuchen aller Transkripte innerhalb eines Projekts
- **Meeting-spezifische Suche:** Suche innerhalb eines bestimmten Meetings
- **Projekt-spezifische Suche:** Suche über alle Meetings eines Projekts
- **Zeitbasierte Suche:** Suche nach Zeiträumen
- **Sprecher-spezifische Suche:** Suche nach Äußerungen bestimmter Sprecher
- **Erweiterte Filter:** Suche nach Audio-Typ, Sprache, Sprecher, etc.

6. Sprecher-Erkennung (Speaker Diarization)

- **Automatische Sprecher-Identifikation:** OpenAI Whisper API mit Diarization
- **Sprecher-Profile:** Verwaltung von Sprecher-Namen und Rollen pro Projekt
- **Sprechzeit-Analyse:** Tracking der Sprechzeiten pro Teilnehmer
- **Sprecher-Zuordnung:** Automatische Zuordnung von Text-Segmenten zu Sprechern

- **Sprecher-Metadaten:** Speicherung von Stimmeigenschaften und Kontext
- **Intelligente Namensgebung:** Auto-Generierung von Sprecher-Namen basierend auf Kontext

7. Datenverwaltung

- **Strukturierte Speicherung:** Organisierte Ablage in relationaler Datenbank
- **Backup-Funktionalität:** Sichern und Wiederherstellen von Daten
- **Datenexport:** Export von Transkripten in verschiedenen Formaten
- **Datenlöschung:** Selektive Löschung von Transkripten
- **Projekt-Isolation:** Vollständige Datenisolation zwischen Projekten

🔌 API-Spezifikation

Authentifizierung

- **API-Key-basierte Authentifizierung** für alle Endpunkte
- **Header-basierte Übertragung** des API-Keys
- **Sichere Übertragung** über HTTPS (Produktionsumgebung)
- **Projekt-basierte Autorisierung** für alle Ressourcen

Core APIs

Projekt-Integration APIs

```
GET /projects
- Liefert Liste aller verfügbaren Projekte aus externem System
- Headers: Authorization (für externe API)
- Optional: Pagination, Filter
- Response: Array von Projekt-Objekten

GET /projects/{id}
- Liefert spezifisches Projekt mit allen Meetings
- Headers: Authorization (für externe API)
- Response: Projekt-Objekt mit Meetings-Array

GET /projects/{id}/validate
- Validiert Projekt-Berechtigung für aktuellen Benutzer
- Headers: Authorization (für externe API)
- Response: { valid: boolean, permissions: string[] }

POST /projects/{id}/sync
- Synchronisiert Projekt-Daten mit externem System
- Headers: Authorization (für externe API)
- Response: Synchronisationsbericht

GET /projects/{id}/export
- Exportiert alle Meeting-Daten des Projekts
- Optional: format (json, csv, pdf)
- Response: Export-Datei
```

Meeting-Management APIs

```
POST /projects/{project_id}/meetings
- Erstellt ein neues Meeting im Projekt
- Payload: { title, description, type: 'planned' | 'spontaneous' }
- Response: Meeting-Objekt mit UUID

POST /projects/{project_id}/meetings/spontaneous
- Erstellt spontanes Meeting beim Start der Aufnahme
- Payload: { audio_type, initial_metadata }
- Response: Meeting-Objekt mit auto-generiertem Titel

GET /projects/{project_id}/meetings
- Liefert Liste aller Meetings des Projekts
- Optional: Pagination, Filter, type (planned/spontaneous)
- Response: Array von Meeting-Objekten

GET /projects/{project_id}/meetings/{id}
- Liefert spezifisches Meeting mit allen Transkripten
- Response: Meeting-Objekt mit Transcripts-Array

PUT /projects/{project_id}/meetings/{id}
- Aktualisiert Meeting-Metadaten
- Payload: { title, description, type }
- Response: Aktualisiertes Meeting-Objekt

POST /projects/{project_id}/meetings/merge
- Führt mehrere Meetings des Projekts zusammen
- Payload: { source_meeting_ids: [], target_meeting_id, merge_strategy }
- Response: Zusammenführungsbericht

GET /projects/{project_id}/meetings/suggestions
- Liefert Vorschläge für Meeting-Zusammenführungen im Projekt
- Optional: time_threshold, similarity_threshold
- Response: Array von Zusammenführungsvorschlägen

DELETE /projects/{project_id}/meetings/{id}
- Löscht Meeting und alle zugehörigen Transkripte
- Response: Bestätigungsnachricht
```

Screenity-Integration APIs

```
POST /projects/{project_id}/screenity/start
- Startet Screenity-Aufnahme für ein Meeting
- Payload: { meeting_id, recording_options }
- Response: Aufnahme-Status und Session-ID

POST /projects/{project_id}/screenity/stop
- Stoppt Screenity-Aufnahme
- Payload: { session_id }
- Response: Aufnahme-Metadaten und Datei-Pfad

GET /projects/{project_id}/screenity/status/{session_id}
- Liefert Status einer laufenden Aufnahme
- Response: Aufnahme-Status und Metadaten

POST /projects/{project_id}/screenity/upload
- Lädt Screenity-Aufnahme hoch
- Multipart form data mit Video/Audio-Datei
```

```
- Headers: meeting-id, session-id, recording-metadata
- Response: Upload-Status und Datei-Referenz
```

Transkription APIs

```
POST /projects/{project_id}/whisper
- Verarbeitet Audio-Daten und erstellt Transkript
- Multipart form data mit Audio-Datei
- Headers: meeting-id, audio-type, chunk-metadata
- Response: Transkript-Text und Speicherbestätigung

POST /projects/{project_id}/whisper/diarization
- Verarbeitet Audio mit Sprecher-Erkennung (Speaker Diarization)
- Multipart form data mit Audio-Datei
- Headers: meeting-id, audio-type, chunk-metadata
- Response: Transkript mit Sprecher-Zuordnung und Sprecher-Profiles

GET /projects/{project_id}/transcripts/{meeting-id}
- Liefert alle Transkripte eines Meetings
- Optional: Filter nach audio-type, Zeitraum, speaker-id
- Response: Array von Transcript-Objekten

GET /projects/{project_id}/speakers/{meeting-id}
- Liefert alle Sprecher-Profile eines Meetings
- Response: Array von Speaker-Objekten mit Metadaten

PUT /projects/{project_id}/speakers/{meeting-id}/{speaker-id}
- Aktualisiert Sprecher-Profile (Name, Rolle, etc.)
- Payload: { name, role, notes }
- Response: Aktualisiertes Speaker-Objekt

DELETE /projects/{project_id}/meetings/{id}/transcripts
- Löscht alle Transkripte eines Meetings
- Response: Anzahl gelöschter Transkripte
```

Such-APIs

```
GET /projects/{project_id}/search
- Durchsucht alle Transkripte des Projekts
- Query-Parameter: q (Suchbegriff), meetingId (optional)
- Response: Array von Suchergebnissen mit Kontext

GET /projects/{project_id}/search/advanced
- Erweiterte Suche mit Filtern innerhalb des Projekts
- Parameter: query, meetingId, audioType, dateRange, language
- Response: Gefilterte Suchergebnisse

GET /search/global
- Durchsucht alle Projekte (nur für Admin-Benutzer)
- Query-Parameter: q (Suchbegriff), projectId (optional)
- Response: Array von Suchergebnissen mit Projekt-Kontext
```

System-APIs

```
GET /health
- System-Status und Verfügbarkeit
- Response: Health-Status und Metriken

```

```
GET /stats
- System-Statistiken
- Response: Anzahl Meetings, Transkripte, etc.
```

🗄️ Datenmodell

Project Entity

```typescript
interface Project {
  id: UUID; // Externe Projekt-ID
  name: string;
  description?: string;
  external_system: string; // Quelle der Projekt-Daten
  external_id: string; // ID im externen System
  created_at: Timestamp;
  updated_at: Timestamp;
  last_synced: Timestamp; // Letzte Synchronisation
  status: 'active' | 'archived' | 'deleted';
  permissions: string[]; // Benutzer-Berechtigungen für dieses Projekt
  metadata?: {
    owner?: string;
    team_members?: string[];
    tags?: string[];
    external_metadata?: any; // Zusätzliche Daten vom externen System
  };
}
```

Meeting Entity

```typescript
interface Meeting {
  id: UUID;
  project_id: UUID; // Referenz zum Projekt
  title: string;
  description?: string;
  type: 'planned' | 'spontaneous';
  created_at: Timestamp;
  updated_at: Timestamp;
  status: 'active' | 'archived' | 'deleted' | 'merged';
  metadata?: {
    participants?: string[];
    duration?: number;
    language?: string;
    auto_generated_title?: boolean;
    original_meeting_ids?: UUID[]; // Für zusammengeführte Meetings
    merge_history?: MergeEvent[];
  };
}
```

```typescript
interface MergeEvent {
  timestamp: Timestamp;
  source_meeting_ids: UUID[];
  target_meeting_id: UUID;
  merge_reason?: string;
}
```

Transcript Entity
```typescript
interface Transcript {
  id: UUID;
  meeting_id: UUID;
  audio_type: 'tab' | 'microphone';
  transcript: string;
  audio_file?: string;
  audio_size: number;
  chunk_start_time: Timestamp;
  chunk_end_time: Timestamp;
  chunk_duration: number;
  language: string;
  confidence_score?: number;
  created_at: Timestamp;
  speaker_segments?: SpeakerSegment[];
  metadata?: {
    speaker_detection?: boolean;
    noise_level?: number;
    quality_score?: number;
    diarization_enabled?: boolean;
  };
}

interface SpeakerSegment {
  speaker_id: string;
  start_time: number; // in Sekunden
  end_time: number;   // in Sekunden
  text: string;
  confidence: number;
}
```

Recording Entity

```typescript
interface Recording {
  id: UUID;
  meeting_id: UUID;
  session_id: string; // Screenity Session ID
  recording_type: 'screen' | 'audio' | 'both';
  video_file?: string;
  audio_file?: string;
  file_size: number;
  duration: number;
  format: 'mp4' | 'webm' | 'gif';
  resolution?: string;
  fps?: number;
  created_at: Timestamp;
  metadata?: {
    annotations?: boolean;
    cursor_highlighting?: boolean;
    zoom_level?: number;
    blur_applied?: boolean;
    screenity_version?: string;
  };
}
```

Speaker Entity

```typescript
interface Speaker {
  id: string; // OpenAI Speaker ID (z.B. "speaker_0", "speaker_1")
  meeting_id: UUID;
  name?: string; // Benutzer-definierter Name
  role?: string; // Rolle im Meeting (z.B. "Moderator", "Teilnehmer")
  notes?: string; // Zusätzliche Notizen
  total_speech_time: number; // Gesamte Sprechzeit in Sekunden
  segments_count: number; // Anzahl der Sprechsegmente
  first_seen: Timestamp;
  last_seen: Timestamp;
  created_at: Timestamp;
  updated_at: Timestamp;
  metadata?: {
    voice_characteristics?: {
      pitch?: number;
      speaking_rate?: number;
      accent?: string;
    };
    ai_generated_name?: string; // Auto-generierter Name basierend auf Kontext
  };
}
```

Search Result

```typescript
interface SearchResult {
  project_id: UUID;
  project_name: string;
  meeting_id: UUID;
  meeting_title: string;
  transcript_id: UUID;
  audio_type: string;
  transcript: string;
  chunk_start_time: Timestamp;
  chunk_end_time: Timestamp;
  created_at: Timestamp;
  relevance_score?: number;
  context_before?: string;
  context_after?: string;
}
```

🔧 Technische Anforderungen

Screenity-Integration

Screenity Features Integration:

```typescript
// Screenity Recording Options
interface ScreenityRecordingOptions {
  // Aufnahme-Typen
  recordingType: 'tab' | 'desktop' | 'camera' | 'area';

  // Audio-Optionen
  audio: {
    microphone: boolean;
    systemAudio: boolean;
```

```
    pushToTalk: boolean;
  };

  // Video-Optionen
  video: {
    resolution: string;
    fps: number;
    quality: 'low' | 'medium' | 'high';
  };

  // Annotation-Optionen
  annotations: {
    enabled: boolean;
    drawing: boolean;
    text: boolean;
    shapes: boolean;
    arrows: boolean;
  };

  // Cursor-Optionen
  cursor: {
    highlightClicks: boolean;
    spotlightMode: boolean;
    showCursor: boolean;
  };

  // Zoom und Blur
  effects: {
    zoomEnabled: boolean;
    blurEnabled: boolean;
    aiBackground: boolean;
  };

  // Timer und Automatisierung
  automation: {
    countdown: number;
    autoStop: number;
    alarms: boolean;
  };
}
```

Screenity API Integration:

```
// Screenity API Interface
interface ScreenityAPI {
  // Aufnahme-Steuerung
  startRecording(options: ScreenityRecordingOptions): Promise<string>; // Session ID
  stopRecording(sessionId: string): Promise<RecordingResult>;
  pauseRecording(sessionId: string): Promise<void>;
  resumeRecording(sessionId: string): Promise<void>;

  // Status und Metadaten
  getRecordingStatus(sessionId: string): Promise<RecordingStatus>;
  getRecordingMetadata(sessionId: string): Promise<RecordingMetadata>;

  // Export und Verarbeitung
  exportRecording(sessionId: string, format: 'mp4' | 'webm' | 'gif'): Promise<Blob>;
  extractAudio(sessionId: string): Promise<Blob>;
```

```

  // Annotation-Management
  saveAnnotations(sessionId: string, annotations: Annotation[]): Promise<void>;
  loadAnnotations(sessionId: string): Promise<Annotation[]>;
}
```

Externe System-Integration

Externe API-Integration:

```
// Beispiel für externe Projekt-API
interface ExternalProjectAPI {
  // Projekt-Abfragen
  getProjects(userId: string): Promise<Project[]>;
  getProject(projectId: string): Promise<Project>;
  validateProjectAccess(userId: string, projectId: string): Promise<boolean>;

  // User-Management
  getUser(userId: string): Promise<User>;
  validateUser(userId: string): Promise<boolean>;

  // Berechtigungen
  getUserPermissions(userId: string, projectId: string): Promise<string[]>;
}
```

Caching-Strategie:

- **Projekt-Cache:** 15 Minuten TTL für Projekt-Metadaten
- **User-Cache:** 30 Minuten TTL für User-Daten
- **Permission-Cache:** 5 Minuten TTL für Berechtigungen
- **Cache-Invalidation:** Bei Änderungen im externen System

Synchronisation:

- **Regelmäßige Sync:** Alle 30 Minuten für Projekt-Updates
- **Event-basierte Sync:** Bei spezifischen Änderungen
- **Incremental Updates:** Nur geänderte Daten synchronisieren
- **Conflict Resolution:** Strategie für Datenkonflikte

Speaker Diarization Integration

Option 1: OpenAI Whisper API (Kommerziell)

- **API-Verfügbarkeit:** OpenAI Whisper API mit Diarization seit 2023 verfügbar
- **Kosten:** Zusätzliche Kosten für Diarization (ca. $0.006 pro Minute Audio)
- **Unterstützte Sprachen:** Alle von Whisper unterstützten Sprachen
- **Audio-Formate:** MP3, MP4, Mpeg, MPGA, M4A, WAV, WEBM
- **Vorteile:** Einfache Integration, hohe Qualität, keine lokale Infrastruktur
- **Nachteile:** Kosten, Abhängigkeit von externem Service

Option 2: Pyannote.audio (Open Source)

- **Verfügbarkeit:** Open Source auf Hugging Face verfügbar
- **Kosten:** Kostenlos (nur Compute-Ressourcen)
- **Lizenz:** MIT License
- **Modelle:** Pyannote/speaker-diarization-3.1, Pyannote/speaker-diarization-3.0
- **Vorteile:** Kostenlos, lokal ausführbar, sehr hohe Qualität
- **Nachteile:** Höhere Compute-Anforderungen, komplexere Integration

Option 3: NeMo Speaker Diarization (Open Source)

- **Verfügbarkeit:** NVIDIA NeMo Toolkit (Open Source)
- **Kosten:** Kostenlos
- **Lizenz:** Apache 2.0
- **Modelle:** Pre-trained models für verschiedene Sprachen
- **Vorteile:** Optimiert für NVIDIA GPUs, gute Performance
- **Nachteile:** GPU-Anforderung für optimale Performance

Option 4: SpeechBrain (Open Source)

- **Verfügbarkeit:** Open Source Python Library
- **Kosten:** Kostenlos
- **Lizenz:** Apache 2.0
- **Modelle:** speechbrain/speaker-diarization-crdnn-libriparty
- **Vorteile:** Einfache Integration, gute Dokumentation
- **Nachteile:** Mittlere Qualität im Vergleich zu Pyannote

Auswahlkriterien:

Für Entwicklung/Prototyping:

- **OpenAI API:** Schnellste Integration, keine lokale Infrastruktur
- **SpeechBrain:** Einfachste Open Source Integration

Für Produktionsumgebung:

- **Pyannote.audio:** Beste Open Source Qualität, kostenlos
- **NeMo:** Optimiert für GPU-Umgebungen, gute Performance

Für Ressourcen-beschränkte Umgebungen:

- **OpenAI API:** Keine lokalen Compute-Ressourcen nötig
- **SpeechBrain:** Niedrigste CPU-Anforderungen

Für Datenschutz-kritische Anwendungen:

- **Pyannote.audio:** Vollständig lokal ausführbar

- **NeMo:** Lokale Verarbeitung, keine externen API-Calls

API-Parameter für OpenAI Diarization:

```
{
  model: 'whisper-1',
  file: audioFile,
  response_format: 'verbose_json',
  language: 'de',
  temperature: 0.0,
  prompt: 'Transkribiere präzise auf Deutsch.',
  // Diarization-spezifische Parameter
  diarization: true,
  diarization_config: {
    min_speakers: 1,
    max_speakers: 10,
    speaker_labels: true
  }
}
```

Pyannote.audio Integration:

```
from pyannote.audio import Pipeline
from pyannote.audio.pipelines.utils.hook import ProgressHook

# Pipeline initialisieren
pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="YOUR_HUGGING_FACE_TOKEN"
)

# Diarization ausführen
with ProgressHook() as hook:
    diarization = pipeline(audio_file, hook=hook)

# Sprecher-Segmente extrahieren
for turn, _, speaker in diarization.itertracks(yield_label=True):
    print(f"Speaker {speaker}: {turn.start:.1f}s - {turn.end:.1f}s")
```

NeMo Integration:

```
from nemo.collections.asr.models import ClusteringDiarizer

# Model laden
diarizer = ClusteringDiarizer.from_pretrained(
    model_name="titanet-l"
)

# Diarization ausführen
diarization_results = diarizer.diarize(
    paths2audio_files=[audio_file],
    num_speakers=2
)
```

Response-Format mit Diarization:

```
{
  "text": "Vollständiger Transkript-Text",
```

```
  "segments": [
    {
      "id": 0,
      "seek": 0,
      "start": 0.0,
      "end": 3.0,
      "text": "Hallo, willkommen zum Meeting.",
      "tokens": [...],
      "temperature": 0.0,
      "avg_logprob": -0.5,
      "compression_ratio": 1.2,
      "no_speech_prob": 0.1,
      "speaker": "speaker_0"
    }
  ],
  "speakers": [
    {
      "id": "speaker_0",
      "name": null,
      "segments": [...],
      "total_speech_time": 45.2
    }
  ]
}
```

Implementierungsstrategie:

Für OpenAI API:

1. **Selektive Aktivierung:** Diarization nur für längere Audio-Clips (>30 Sekunden)
2. **Hybrid-Ansatz:** Normale Transkription für kurze Clips, Diarization für längere
3. **Sprecher-Konsistenz:** Tracking von Sprecher-IDs über mehrere Chunks hinweg
4. **Fallback-Mechanismus:** Normale Transkription bei Diarization-Fehlern

Für Open Source Lösungen:

1. **Docker-Container:** Pyannote.audio oder NeMo in separatem Container
2. **Microservice-Architektur:** Diarization als separater Service
3. **Batch-Verarbeitung:** Diarization für komplette Meetings nach Aufnahme
4. **Caching-Strategie:** Vorverarbeitete Sprecher-Profile für wiederkehrende Teilnehmer
5. **Ressourcen-Management:** GPU-Nutzung für NeMo, CPU für Pyannote

Chrome Extension

- **Manifest V3** kompatibel
- **Screenity Integration** für Screen- und Audio-Aufnahme
- **Projekt-Management Interface** für Meeting-Verwaltung
- **Background Scripts** für Session-Management
- **Popup Interface** für Benutzer-Interaktion
- **Offscreen Document** für Audio-Extraktion und -Verarbeitung

Backend-Server

- **Node.js** mit Express.js
- **PostgreSQL** als Hauptdatenbank
- **Multer** für File-Upload-Handling
- **OpenAI Whisper API** Integration mit Speaker Diarization
- **CORS** Konfiguration für Cross-Origin Requests
- **Rate Limiting** für API-Schutz
- **Externe API-Integration** für Projekt- und User-Management

Speaker Diarization Service (Optional)

- **Python Microservice** für Open Source Diarization
- **Docker Container** für Pyannote.audio oder NeMo
- **GPU Support** für NeMo (optional, CPU-Fallback verfügbar)
- **Hugging Face Integration** für Pyannote Modelle
- **Batch Processing** für effiziente Verarbeitung

Externe System-Integration

- **API-Client** für externe Projekt-Management-Systeme
- **Authentication Proxy** für externe Authentifizierung
- **Caching Layer** für Projekt- und User-Daten
- **Synchronisation Service** für regelmäßige Daten-Updates

Screenity-Integration

- **Vollständige Screenity-Funktionalität** - Alle Features von Screenity
- **Video-Formate:** MP4, WebM, GIF Export
- **Audio-Extraktion** aus Video-Dateien für Transkription
- **Annotation-Integration** - Zeichnungen und Markierungen
- **Cursor-Tracking** und Highlighting
- **Zoom- und Blur-Funktionen**
- **Offline-Speicherung** mit IndexedDB

Audio-Verarbeitung

- **Audio-Extraktion** aus Screenity-Videos
- **WebM/MP3** Format-Unterstützung
- **Chunk-basierte Verarbeitung** (10-20 Sekunden)
- **Stille-Erkennung** für optimale Chunk-Größen

- **Rauschunterdrückung** für bessere Transkriptionsqualität
- **Audio-Normalisierung** für konsistente Lautstärke

Datenbank

- **PostgreSQL** mit UUID-Primary-Keys
- **Indizes** für effiziente Suche
- **Foreign Key Constraints** für Datenintegrität
- **JSON-Felder** für flexible Metadaten
- **Partitionierung** für große Datenmengen (optional)

🎨 Benutzer-Interface Anforderungen

Chrome Extension Popup

- **Projekt-Auswahl:** Dropdown oder Liste verfügbarer Projekte aus externem System
- **Meeting-Auswahl:** Dropdown oder Liste verfügbarer Meetings im ausgewählten Projekt
- **Screenity-Integration:** Direkte Integration des Screenity Recorders
- **Schnellstart:** Direkte Screenity-Aufnahme ohne Meeting-Auswahl (erstellt spontanes Meeting im ausgewählten Projekt)
- **Aufnahme-Steuerung:** Screenity-Controls mit Status-Anzeige
- **Live-Feedback:** Aktuelle Aufnahme-Zeit und Screenity-Status
- **Meeting-Verwaltung:** Anzeige und Bearbeitung von spontanen Meetings
- **Zusammenführungsvorschläge:** Automatische Vorschläge für ähnliche Meetings im Projekt
- **Einstellungen:** Konfiguration von Backend-URL, API-Key, externer API-URL, Screenity-Einstellungen, etc.

Erweiterte UI-Features (Optional)

- **Projekt-Synchronisation:** Interface zur Synchronisation mit externem System
- **Meeting-Dashboard:** Übersicht aller Meetings eines Projekts mit Statistiken
- **Screenity-Editor:** Integration des Screenity Video-Editors für Nachbearbeitung
- **Transkript-Viewer:** Anzeige und Bearbeitung von Transkripten mit Sprecher-Zuordnung
- **Sprecher-Verwaltung:** Interface zur Bearbeitung von Sprecher-Profilen und Namen pro Projekt
- **Such-Interface:** Benutzerfreundliche Suchfunktion mit Projekt- und Sprecher-Filter
- **Export-Funktionen:** Download von Videos, Transkripten und Meeting-Daten in verschiedenen Formaten

- **Meeting-Zusammenführung:** Interface für das Zusammenführen von Meetings innerhalb eines Projekts
- **Spontane Meetings:** Verwaltung und Umbenennung von spontanen Meetings
- **Meeting-Timeline:** Chronologische Ansicht aller Meetings eines Projekts mit Zusammenführungsvorschlägen
- **Sprecher-Analytics:** Statistiken zu Sprechzeiten und Beteiligung der Teilnehmer pro Projekt
- **Video-Player:** Integrierter Player für Screenity-Aufnahmen mit Transkript-Synchronisation

🔒 Sicherheitsanforderungen

Authentifizierung & Autorisierung

- **API-Key-basierte Authentifizierung** für alle Endpunkte
- **Projekt-basierte Autorisierung** für alle Ressourcen
- **Rate Limiting** zur Verhinderung von Missbrauch
- **Input-Validierung** für alle API-Parameter
- **SQL-Injection-Schutz** durch Parameterized Queries
- **Tenant-Isolation** - Vollständige Datenisolation zwischen Projekten

Datenschutz

- **Lokale Audio-Verarbeitung** wo möglich
- **Verschlüsselte Übertragung** (HTTPS)
- **Datenminimierung** - nur notwendige Daten speichern
- **Löschfunktionen** für Benutzer-Daten
- **Projekt-basierte Datenisolation** - Keine Datenlecks zwischen Projekten
- **Granulare Berechtigungen** pro Projekt

Audit & Logging

- **API-Access-Logs** für Debugging und Monitoring
- **Error-Logging** für System-Diagnose
- **Performance-Monitoring** für Optimierung

📊 Performance-Anforderungen

Latenz

- **API-Response-Zeit:** < 200ms für CRUD-Operationen
- **Transkription-Zeit:** < 30 Sekunden für 20-Sekunden-Audio
- **Such-Response-Zeit:** < 500ms für Standard-Suchen

Skalierbarkeit

- **Gleichzeitige Meetings:** Unterstützung für 100+ aktive Meetings
- **Datenbank-Performance:** Optimierte Indizes für große Transkript-Mengen
- **Audio-Verarbeitung:** Effiziente Chunk-basierte Verarbeitung

Verfügbarkeit

- **Uptime:** 99.9% Verfügbarkeit
- **Backup-Strategie:** Regelmäßige Datenbank-Backups
- **Error-Handling:** Graceful Degradation bei API-Fehlern

🚀 Deployment & Infrastruktur

Entwicklungsumgebung

- **Docker Compose** für lokale Entwicklung
- **Hot Reload** für Backend-Entwicklung
- **Extension-Reload** für Chrome Extension Testing

Produktionsumgebung

- **Container-basierte Deployment** (Docker)
- **Load Balancer** für horizontale Skalierung
- **CDN** für statische Assets
- **Monitoring & Alerting** für System-Überwachung

📈 Zukünftige Erweiterungen

Phase 2 Features

- **Real-time Collaboration:** Mehrere Benutzer pro Meeting
- **Speaker Recognition:** Automatische Sprecher-Identifikation
- **Meeting Analytics:** Detaillierte Statistiken und Insights
- **Integration APIs:** Anbindung an Kalender- und Meeting-Systeme
- **Intelligente Meeting-Erkennung:** Automatische Erkennung von Meeting-Kontexten
- **Meeting-Templates:** Vorlagen für häufige Meeting-Typen
- **Automatische Zusammenführung:** KI-basierte Vorschläge für Meeting-Zusammenführungen

Phase 3 Features

- **AI-basierte Zusammenfassung:** Automatische Meeting-Zusammenfassungen
- **Action Item Extraction:** Automatische Erkennung von To-Dos

- **Sentiment Analysis:** Stimmungsanalyse der Meeting-Teilnehmer
- **Multi-Platform Support:** Mobile Apps und Desktop-Anwendungen

---

Diese Spezifikation dient als Grundlage für die Entwicklung und kann je nach Anforderungen und Feedback angepasst werden.